

코딩은 Claude와 함께, 배운 것은 내 것으로.

learn-notes는 Claude가 고친 코드가 무엇을 어떻게 달라지게 했는지 턴마다 학습 노트로 보여 주고, 잊을 때쯤 내가 만든 코드로 복습 퀴즈를 내 주는 Claude Code 모드입니다. 모드(MOD)가 무엇인지부터 설치, 기능, 팀에서 쓰는 법까지 한 번에 안내합니다.

버전 1.6.0

2026년 10월

github.com/Owen-YC/learn-notes

claude - ~/shop

전체 화면 · 200x50 · 패널: ctrl+x tab으로 키 · 1~4 보기 · /learn 복습

> 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘

• Read(orders.js)

└ Read 12 lines

• Update(orders.js)

└ Updated orders.js with 4 additions and 4 removals

- 반복문을 for...of로 바꾸고 minAmount 인자(기본값 0)를 추가해 그 금액 이상인 결제 주문만 셉니다.

노트 2/2 · 05:12 · 파일 1개 · +4 -4

p: ◀ 이전 n: 다음 ▶

요청: 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘

1: 노트 · 2: 전/후 · 3: 개념 모음 · 4: 퀴즈 • q: 복습 1개

Esc: 대화로 돌아가기 · 글자 키는 영문 상태에서

한 줄 요약

결제된 주문을 세던 함수가 이제 최소 금액(minAmount) 이상인 결제 주문만 셉니다.

무엇이 바뀌었나

orders.js

- 전 결제(paid)된 주문의 개수를 셉니다. 인덱스(orders[i])로 주문을 하나씩 확인합니다.

+ 후 결제되었고 금액(amount)이 minAmount 이상인 주문만 셉니다. 인자를 주지 않으면 0이 기본값이 되어, 금액이 0 이상인 주문은 모두 셉니다.

→ 예 countPaid(orders, 10000), 결제 주문 금액이 5,000원과 20,000원일 때 → 전: 2 / 후: 1

왜 이렇게 바꿨을까

요청하신 최소 금액 조건을 넣으려고 minAmount 인자를 추가했습니다. 기본값을 0으로 두어서, 기존처럼 두 번째 인자 없이 부르는 코드는 결과가 바뀌지 않게 했습니다. 반복문은 인덱스를 세는 대신 주문을 바로 꺼내는 방식으로 바꿔 읽기 쉽게 했습니다. 아마 orders[i]를 여러 번 쓰던 번거로움을 줄이려는 의도일 것입니다.

배울 개념

- **for...of (복습)**: 배열의 값을 하나씩 꺼내 쓰는 반복문입니다. 순번(인덱스)을 따로 세지 않아도 됩니다 - for (const order of orders) {
- **기본값 매개변수 (복습)**: 인자를 생략했을 때 쓸 값을 미리 정해두는 문법입니다. 여기서는 0입니다 - function countPaid(orders, minAmount = 0) {
- **복합 대입 연산자**: 현재 값에 더한 결과를 변수에 바로 저장하는 줄임 표기입니다. n += 1은 n = n + 1과 같습니다 - n += 1

직접 확인해 볼 것

- countPaid(orders)와 countPaid(orders, 0)의 결과가 같은지 실행해 보세요. 기본값이 실제로 0으로 쓰이는지 알 수 있습니다.
- amount가 없는 주문을 넣어 보세요. undefined >= 0은 false라서 그 주문은 세지 않습니다. 금액 필드가 빠진 데이터가 있을 때 어떤 결과가 나오는지 확인할 수 있습니다.

새로 배운 개념 복합 대입 연산자

다시 만난 개념 for...of x2 · 기본값 매개변수 x2

이 노트로 t: 이 노트 퀴즈 e: 더 쉽게 r: 예시로 따라가기

i: 질문하기 w: 다시 쓰기

질문 [궁금한 것을 적고 Enter]

들어가며

15분이면 다 읽고, 오늘부터 바로 쓸 수 있습니다.

이 안내서는 팀에서 Claude Code로 코딩하는 분들을 위해 썼습니다. 특히 코드를 직접 쓰기보다 Claude에게 맡기는 일이 많은 분, 코딩을 배우는 중인 분께 도움이 됩니다. 처음 듣는 말이 나와도 괜찮습니다. 필요한 용어는 그때그때 풀어 설명합니다.

1

모드(MOD)란 무엇인지

Claude Code에 기능을 더하는 확장이 어떻게 동작하는지 이해합니다.

2

설치하고 첫 노트 보기

두 줄로 설치하고, 첫 요청에서 학습 노트를 받아 봅니다.

3

배운 것을 내 것으로

질문, 퀴즈, 간격 반복 복습, 기록으로 실력을 쌓습니다.

차례

1	왜 learn-notes인가요?	바이브코딩의 장점과 함정
2	Claude Code와 모드(MOD)	개념 · 용어 · 동작 원리
3	설치하고 첫 노트 보기	5단계 · 한/영 키 · Windows 팁
4	학습 노트 읽기	노트 다섯 칸 · 숫자 키 1~4
5	노트에서 바로 묻기	이 노트 퀴즈 · 더 쉽게 · 따라가기 · 대화창에서 묻기
6	내 코드로 푸는 복습 퀴즈	문제 유형 · Enter로 이어 풀기 · 간격 반복 · 줄임
7	쌓이는 기록	개념 모음 · 기록 · 정리 · 보고서
8	명령 한눈에 보기	/learn 하위 명령
9	설정 · 비용 · 개인정보	빠는 파일 · 가리는 값
10	팀으로 쓰기	팀 규칙 파일 · 관리자 기본값 · 하루 한도
11	문제 해결	노트가 안 생길 때 · 키가 안 먹을 때
12	팀에서 이렇게 써 보세요	하루 · 한 주 루틴

1.6.0에서 달라진 것

- 보기는 숫자 **1** ~ **4**, **q** 는 오늘 복습 바로 시작. **v** 는 없어졌습니다.
- **/learn** 복습 한 번이면 패널이 퀴즈로 열리고 답 칸에서 시작합니다.
- 퀴즈는 내 코드로 **예측 · 왜 · 바꿔 보기**. 답을 적고 Enter면 이어집니다.
- 복습 간격은 직접 떠올린 것만 늘고, 오늘 복습은 하루 10개까지입니다.
- **.env** · 잠금 파일은 모으지 않고, 흔한 키는 **«가림»** 으로 가립니다.
- 새로: **/learn 보고서**, 대화창에 그냥 묻기, 팀 규칙 파일.

1 왜 learn-notes인가요?

코딩 속도는 그대로, 배우는 것은 덤으로.

Claude에게 "결제된 주문만 세는 함수에 최소 금액 조건도 넣어 줘"라고 말하면 몇 초 만에 코드가 바뀝니다. 이렇게 AI에게 코드를 맡기며 만드는 방식을 흔히 **바이브코딩**이라고 부릅니다. 빠르고 편하지만, 하루가 끝나고 돌아보면 "무엇이 왜 바뀌었는지"는 머리에 거의 남지 않습니다.



왜 바뀌었는지 모릅니다

Claude의 설명은 대화 속에 묻히고, 바뀐 코드의 전후를 다시 찾아보기 어렵습니다.



같은 개념을 매번 처음 봅니다

지난주에 본 `for...of` 를 오늘 또 처음 보는 것처럼 지나칩니다.



배운 것이 남지 않습니다

무엇을 배웠는지 기록이 없어, 성장했는지 스스로 알기 어렵습니다.

learn-notes는 이 세 가지를 Claude Code 안에서 바로 해결합니다. 따로 공부 시간을 내지 않아도, 평소처럼 코딩을 요청하기만 하면 됩니다.



언마다 학습 노트

Claude가 파일을 고친 턴이 끝나면, 무엇이 왜 바뀌었고 무엇을 배울 수 있는지 노트로 정리합니다.



잊을 때쯤 복습 퀴즈

배운 개념을 모아 두었다가, 잊을 때쯤 내가 만든 코드로 문제를 냅니다. 답은 Claude가 채점합니다.



쌓이는 기록

날짜별 일지, 개념 모음, 연속 학습일과 정답률이 쌓여 성장이 눈에 보입니다.

"Claude가 코드를 고칠 때마다, 나에게는 짧은 수업이 하나씩 남습니다."

claude -- /shop

전체 화면 · 200x50 · 패널: ctrl+x tab으로 키 · 1-4 보기 · /learn 복습

> 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘

- Read(`orders.js`)
 - Read 12 lines
- Update(`orders.js`)
 - Updated `orders.js` with 4 additions and 4 removals

* Responding...

- 작업 중: 파일 1개 · 턴이 끝나면 노트를 씁니다
 - `orders.js` +4 -4

노트 1/1 · 지난 세션 · 10-01 06:12 · 파일 1개 · +5 -5

p: ◀ 이전 n: 다음 ▶

요청: `cart.js`의 `total` 함수를 부르면 에러가 나. 고쳐주고, 확인...

1: 노트 · 2: 전/후 · 3: 개념 모음 · 4: 퀴즈 • q: 복습 3개

Esc: 대화로 돌아가기 · 글자 키는 영문 상태에서

한 줄 요약

장바구니 합계 함수가 항상 에러를 내던 반복문을 고치고, 확인율을 받을 수 있게 했습니다.

무엇이 바뀌었나

`cart.js`

- 전: 목록의 끝을 한 칸 넘겨 읽어서 (`i <= items.length`) 합계를 구하지 못하고 에러가 났습니다.
- + 후: 목록의 모든 항목을 한 번씩 더해 합계를 구하고, 확인율(기본값 0)을 곱해 돌려줍니다.

→ 예 `total([ { price: 100, qty: 2 } ], 0.1)` → 전: 에러 / 후: 180

왼쪽은 평소의 Claude Code 대화, 오른쪽이 learn-notes 패널입니다. Claude가 일하는 동안 바뀌는 파일이 패널 맨 위에 실시간으로 쌓입니다.

2 Claude Code와 모드(MOD)

모드는 Claude Code에 끼우는 작은 확장입니다.

Claude Code는

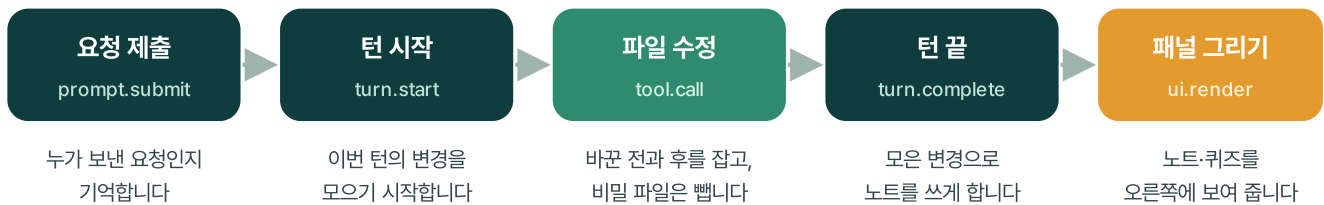
Anthropic이 만든 AI 코딩 도우미입니다. 터미널(또는 데스크톱 앱)에서 Claude와 대화하면, Claude가 파일을 읽고 코드를 고치고 명령을 실행합니다. 내가 요청을 보내고 Claude가 일을 마치고 답하기까지를 **턴(turn)** 하나라고 부릅니다.

모드(MOD)는

스마트폰에 앱을 설치하면 새 기능이 생기듯, **모드를 설치하면 Claude Code에 새 화면과 동작이 생깁니다.** 게임의 "모드"처럼 원래 프로그램은 그대로 두고 그 위에 기능을 얹는다고 생각하면 쉽습니다. Claude가 일하는 방식은 바뀌지 않고, 모드는 옆에서 지켜보다가 필요한 것만 채깁니다.

모드는 이렇게 동작합니다

Claude Code의 한 턴은 몇 가지 이벤트로 이어집니다. learn-notes의 **훅**이 각 이벤트에서 하는 일은 아래와 같습니다.



턴이 끝나면

작은 모델(기본 haiku)이 비밀값을 가린 변경과 요청을 읽고 노트를 씁니다 → 패널에 표시 · 일지에 저장 · 개념 모음에 더하기

알아 두면 좋은 용어

용어	뜻	learn-notes에서는
마켓플레이스	모드(플러그인)를 나눠 주는 목록입니다. 보통 GitHub 저장소 하나입니다.	Owen-YC/learn-notes
플러그인	설치하고 지우는 단위입니다. 설정 항목, 명령, 훅을 한데 담은 꾸러미입니다.	learn-notes@learn-notes
모드 (mod)	플러그인 안에서 훅으로 Claude Code의 화면과 동작을 바꾸는 코드입니다.	노트·패널·퀴즈 코드 전체
이벤트	요청 제출, 파일 수정, 턴 끝처럼 Claude Code 안에서 일어나는 일입니다.	파일이 바뀌는 순간을 압니다
훅 (hook)	특정 이벤트가 일어날 때 끼어들어 실행되는 함수입니다.	바뀌기 전과 후를 잡아 둡니다
패널	대화 오른쪽(또는 프롬프트 위)에 붙는 모드 전용 화면입니다.	학습 노트 패널

모드가 할 수 있는 것, learn-notes가 쓰는 곳



패널

대화 옆 전용 화면. 노트·전/후·개념
모음·퀴즈를 숫자 키로 오깁니다.



슬래시 명령

`/learn` 하나로 퀴즈, 정리, 질문,
기록, 보고서를 엽니다.



상태줄·알림

오늘 복습할 개수를 상태줄에, 노트
완성을 알림으로 알려 줍니다.



작은 모델 호출

노트·퀴즈·채점·질문의 답을 저렴한
모델(haiku)이 씁니다.



저장

일지와 개념 모음은 내 컴퓨터
파일로 남고, 지난 노트와 퀴즈는
다시 켜도 이어집니다.



Claude가 읽는 도구

대화창에 "오늘 배운 거 알려 줘"라고
물으면 Claude가 내 학습 기록을
읽고 답합니다.

안심하세요

모드는 내 컴퓨터의 Claude Code 안에서 돌고, Claude Code가 허락한 기능만 쓸 수 있습니다. learn-notes가 만드는 노트와 기록은 모두 내 컴퓨터에 저장되고, 모델 호출은 지금 쓰는 Claude Code 계정으로 나갑니다. `.env` 같은 비밀 파일은 내용을 모으지 않고, 흔한 키 형식은 가린 뒤 보냅니다(9장).

3

설치하고 첫 노트 보기

준비물: Claude Code v2.1.289 이상 (`claude --version`으로 확인)

1

설치합니다

Claude Code 안에서 두 줄을 입력합니다. 앞 줄은 learn-notes를 나눠 주는 곳(마켓플레이스)을 등록하고, 뒤 줄은 설치합니다.

```
/plugin marketplace add Owen-YC/learn-notes
/plugin install learn-notes@learn-notes
```

2

다시 시작하고 전체 화면으로 바꿉니다

Claude Code를 껐다 켜 뒤 `/tui fullscreen` 을 입력합니다. 패널이 대화 오른쪽에 붙으려면 전체 화면이어야 합니다. 전체 화면이 아니면 프롬프트 위에 열립니다. 처음 켜면 "learn-notes가 켜졌습니다" 알림이 한 번 뜹니다.

3

패널을 엽니다

`/learn` 을 입력합니다. 아직 노트가 없다는 안내와 첫 요청 예시가 보이면 성공입니다. 이때는 키가 대화 입력칸에 그대로 있으니 바로 요청을 적으면 됩니다.

4

평소처럼 코딩을 요청합니다

예: `hello.js`에 이름을 받아 인사하는 함수를 만들어 줘. Claude가 파일을 만들거나 고친 턴이 끝나면 몇 초 안에 노트가 뜹니다.

5

키는 패널로 옮겨서 누릅니다

`/learn`은 패널만 열고 키는 대화 입력칸에 둡니다. 패널의 키를 쓰려면 `ctrl` + `x` 다음 `tab`, 또는 패널을 클릭합니다. 숫자 `1` ~ `4`로 보기를 바꾸고, `Esc`를 누르면 대화로 돌아갑니다(패널은 열린 채). 오늘 복습은 `/learn` 복습 한 번이면 답 칸까지 갑니다.

한/영 키를 확인하세요

숫자 키(`1` ~ `4`)는 한/영 상태와 상관없이 눌립니다. 글자 키(`q` · `w` · `t` · `s` ...)는 **영문 상태**에서 누르세요. 한글 상태면 `q`가 ㅊ으로 들어가 눌리지 않습니다. 답 칸과 질문 칸에는 한글로 그냥 적으면 됩니다.

WINDOWS 사용자

PowerShell에서 `irm https://claude.ai/install.ps1 | iex`로 Claude Code를 설치한 뒤 위와 똑같이 씁니다. Claude Code는 **작업할 폴더에서** 켜세요(`cd ~\practice` 뒤 `claude`). `C:\Windows\System32`에서 켜면 파일이 임시 폴더에 만들어집니다. 그래도 노트는 생기고, 패널에 안내가 뜹니다.

업데이트 · 지우기

업데이트: `/plugin` → `Installed` → `learn-notes` → `Update`, 그다음 `/reload-plugins`. **지우기:** 터미널에서 `claude plugin uninstall learn-notes@learn-notes`. 일지 파일(`~/.claude/learning-notes/`)은 남습니다. **클라우드 세션**(웹·앱)에서는 패널 대신 명령으로 씁니다.

claude -- ~/shop

* 새 세션 · ~/shop

지난 세션의 노트가 패널에 다시 올라와 있습니다

전체 화면 · 200x50 · 패널: ctrl+x tab으로 키 · 1~4 보기 · /Learn 복습

노트 1/1 · 지난 세션 · 10-01 06:12 · 파일 1개 · +5 -5

p: ◀ 이전 n: 다음 ▶

요청: cart.js의 total 함수를 부르면 에러가 나. 고쳐주고, 할인...

1: 노트 · 2: 전/후 · 3: 개념 모음 · 4: 퀴즈 • q: 복습 3개

Esc: 대화로 돌아가기 · 글자 키는 영문 상태에서

한 줄 요약

장바구니 합계 함수가 항상 에러를 내던 반복문을 고치고, 할인을 받을 수 있게 했습니다.

무엇이 바뀌었나

cart.js

- 전: 목록의 끝을 한 칸 넘겨 읽어서(`i <= items.length`) 합계를 구하지 못하고 에러가 났습니다.

+ 후: 목록의 모든 항목을 한 번씩 더해 합계를 구하고, 할인을(기본 값 0)을 곱해 돌려줍니다.

→ 예 `total([{ price: 100, qty: 2 }], 0.1)` → 전: 에러 / 후: 180

왜 이렇게 바꿨을까

에러의 원인은 `i <= items.length`입니다. 목록의 마지막 번호는

`/learn`으로 패널을 연 모습입니다. 노트가 있으면 마지막 노트가 보이고, 지난 세션의 노트도 이어서 볼 수 있습니다. 아직 노트가 없으면 첫 요청 예시가 나옵니다.

학습 노트 읽기

노트는 다섯 칸으로, 350단어 안쪽으로 짧게 씁니다.

칸	무엇이 담기나
한 줄 요약	이번 턴에 무엇을 했는지 한 문장으로
무엇이 바뀌었나	코드가 하던 일(- 전), 이제 하는 일(+ 후), 차이가 드러나는 입력과 결과(→ 예)를 한 줄씩. 결과를 확실히 알 수 없으면 예 줄은 뺍니다
왜 이렇게 바꿨을까	요청과 Claude의 설명, 바뀐 코드를 근거로 이유를 풀어 줍니다. 팀 규칙 파일이 있으면 끝에 <code>팀 규칙: ...</code> 한 줄이 붙기도 합니다 (10장)
배울 개념	1~3개. 짧은 개념 이름, 쉬운 설명, 그 개념이 쓰인 코드 한 줄
직접 확인해 볼 것	직접 실행하거나 바꿔 보며 확인할 것 1~2개

노트 아래에는 이 노트에서 **새로 배운 개념**과 전에 배운 적 있는 **다시 만난 개념**(만난 횟수와 함께)이 나뉘어 나옵니다. 이미 배운 개념은 같은 이름으로 쌓이고, 노트에 "(복습)"이 붙습니다. "아는 개념"으로 표시한 것은 배울 개념에 다시 넣지 않습니다(6장).

설명 수준

설정의 level로 노트의 눈높이를 고릅니다. **beginner**(기본)는 용어를 괄호 안에 쉬운 말로 풀고 비유를 곁들이고, **intermediate**는 개념과 "왜"에, **advanced**는 설계와 트레이드오프에 무게를 둡니다.

숫자 키로 보기 바꾸기 패널 위쪽 줄

패널 위쪽 줄에 1: 노트 · 2: 전/후 · 3: 개념 모음 · 4: 퀴즈 가 있고, 지금 보는 것은 굵게 밑줄이 쳐집니다. 번호는 늘 같습니다. 노트가 없을 때도 1 · 3 · 4 로 번호가 바뀌지 않습니다. 이름을 클릭해도 됩니다.

키	하는 일
1 2 3 4	노트 · 전/후 · 개념 모음 · 퀴즈. 4로 가면 풀 문제가 있을 때 커서가 바로 내 답 칸에 갑니다
q	오늘 복습 바로 시작. 오늘 복습할 개념이 남았으면 보기 줄 끝에 노란 ● 복습 3개 처럼 보입니다. 누르면 퀴즈로 가서 바로 문제를 받습니다(풀던 문제가 있으면 그 문제로)

노트 2/2 · 05:12 · 파일 1개 · +4 -4

p: ◀ 이전 n: 다음 ▶

요청: 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘

1: 노트 · 2: 전/후 · 3: 개념 모음 · 4: 퀴즈 ● q: 복습 1개

Esc: 대화로 돌아가기 · 글자 키는 영문 상태에서

한 줄 요약

결제된 주문을 세던 함수가 이제 최소 금액(minAmount) 이상인 결제 주문만 셉니다.

무엇이 바뀌었나

orders.js

- 전 결제(paid)된 주문의 개수를 셉니다. 인덱스(orders[i])로 주문을 하나씩 확인합니다.

+ 후 결제되었고 금액(amount)이 minAmount 이상인 주문만 셉니다. 인자를 주지 않으면 0이 기본값이 되어, 금액이 0 이상인 주문은 모두 셉니다.

→ 예 countPaid(orders, 10000), 결제 주문 금액이 5,000원과 20,000원일 때 → 전: 2 / 후: 1

왜 이렇게 바꿨을까

요청하신 최소 금액 조건을 넣으려고 minAmount 인자를 추가했습니다. 기본값을 0으로 두어서, 기존처럼 두 번째 인자 없이 부르는 코드는 결과가 바뀌지 않게 했습니다. 반복문은 인덱스를 세는 대신 주문을 바로 꺼내는 방식으로 바꿔 읽기 쉽게 했습니다. 아마 orders[i]를 여러 번 쓰던 번거로움을 줄이려는 의도일 것입니다.

배울 개념

• for...of (복습): 배열의 값을 하나씩 꺼내 쓰는 반복문입니다. 순번(인덱스)을 따로 세지 않아도 됩니다 - for (const order

실제 노트(haiku가 쓴 글)

p n

이전 · 다음 노트(노트 · 전/후 보기에서). 지난 세션의 노트까지 거슬러 올라갑니다

w

노트를 다시 씁니다. 다 쓴 노트에서는 "이 노트로" 줄 끝에 흐리게 있고, 아직 안 쓴 노트에서는 맨 위에 "노트 쓰기"로 있습니다. 다시 쓰다 실패하면 원래 노트는 그대로 둡니다

Esc

대화로 돌아가기. 패널이 키를 받는 동안 흐리게 "Esc: 대화로 돌아가기 · 글자 키는 영문 상태에서"가 보입니다

```
claude - ~/shop

> 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘

• Read(orders.js)
  └ Read 12 lines

• Update(orders.js)
  └ Updated orders.js with 4 additions and 4 removals

• 반복문을 for...of로 바꾸고 minAmount 인자(기본값 0)를 추가해 그 금액 이상인 결제 주문만 셉니다.
```

```
노트 2/2 · 05:12 · 파일 1개 · +4 -4
p: ◀ 이전 n: 다음 ▶
요청: 결제된 주문만 세는 함수에 최소 금액 조건도 넣어줘
1: 노트 · 2: 전/후 · 3: 개념 모음 · 4: 퀴즈 • q: 복습 1개
Esc: 대화로 돌아가기 · 글자 키는 영문 상태에서

orders.js
- 전 결제(paid)된 주문의 개수를 셉니다. 인덱스(orders[i])로 주문을 하나씩 확인합니다.
+ 후 결제되었고 금액(amount)이 minAmount 이상인 주문만 셉니다. 인자를 주지 않으면 0이 기본값이 되어, 금액이 0 이상인 주문은 모두 셉니다.
→ 예 countPaid(orders, 10000), 결제 주문 금액이 5,000원과 20,000원일 때 → 전: 2 / 후: 1

바뀐 줄은 -+로, 그 줄에서 바뀐 낱말은 굵게 표시합니다

orders.js
수정 · +4 -4
- 전 · 3~8행
3 // 결제 완료된 주문 수를 셉다
```

전/후 보기(2). 맨 위에 노트의 전 · 후 · 예가 다시 나오고, 바뀐 곳마다 원래 코드(전)와 바뀐 코드(후)를 줄 번호 그대로 놓습니다. 바뀌지 않은 줄은 흐리게, 바뀐 줄은 -+로, 그 줄에서 바뀐 낱말만 굵은 색으로 칠합니다. 노트에 담지 않은 파일이 있으면 맨 아래에 노트에서 뺀 파일: ...로 이름만 적습니다.

5 노트에서 바로 묻기

노트 맨 아래 "이 노트로" 줄에서 이어 갑니다.

t

이 노트 퀴즈

방금 읽은 노트의 개념으로 문제를 받습니다. 결과를 맞히는 예측과 코드를 바꿔 보는 문제만 내고, 노트의 정의를 그대로 되묻지 않습니다.

e

더 쉽게

원래 노트는 그대로 두고, 노트 아래에 ▶ 더 쉽게로 짧은 문장과 일상의 비유를 덧붙입니다. 나란히 비교해 읽습니다. 설정과 상관없이 입문자 눈높이로 씁니다.



예시로 따라가기

예시 입력 하나로 바뀐 코드를 한 단계씩 따라가며 변수 값이 어떻게 바뀌는지, 바뀌기 전 코드라면 어디서 결과가 갈리는지 보여 줍니다.



질문하기

노트 아래 질문칸에 적고 Enter. 그 노트와 코드를 근거로 짧게 답합니다.

질문칸에 "기본값 0은 왜 넣은 거야? 안 넣으면 어떻게 돼?"처럼 궁금한 것을 그대로 적으면 됩니다. 답은 노트 바로 아래에 남고(최근 2개가 보입니다), 일지 파일에도 기록됩니다.

▶ 더 쉽게와 ▶ 예시로 따라가기도 같은 자리에 남습니다.

이어서 "그럼 음수를 넣으면?"처럼 물으면 **앞의 질문과 답을 읽고** 이어서 답합니다. 명령으로는

`/learn 질문 물을 말` 입니다. 패널에서 고른 노트(없으면 마지막 노트)에 대해 답합니다.

서 그 주문은 세지 않습니다. 금액 필드가 빠진 데이터가 있을 때 어떤 결과가 나오는지 확인할 수 있습니다.

새로 배운 개념 복합 대입 연산자
다시 만난 개념 `for...of` x2 · 기본값 매개변수 x2

이 노트로 t: 이 노트 퀴즈 e: 더 쉽게 r: 예시로 따라가기
i: 질문하기 w: 다시 쓰기

질문 · 기본값 0은 왜 넣은 거야? 안 넣으면 어떻게 돼?
기본값 0은 두 번째 인자를 생략했을 때 쓸 대체값입니다. 커피를 주문할 때 사이즈를 고르지 않으면 "레귤러"로 정해 두는 것과 같습니다. 0을 기본값으로 두면 조건이 `order.amount >= 0`이 되어 금액으로 거르지 않는 것과 같아집니다. 그래서 `countPaid(orders)`를 부르는 기존 코드는 예전과 같은 결과를 냅니다.
기본값이 없으면 `minAmount`가 `undefined`가 됩니다. 그러면 `order.amount >= undefined`는 항상 `false`이므로, 결제된 주문이 있어도 결과는 0이 됩니다. 예러가 나지 않아서 발견하기 어려운 버그입니다.

```
countPaid(orders) // minAmount = 0, 기존과 같은 결과
countPaid(orders, 0) // 위와 같음
```

주의할 점도 있습니다. 이전 코드는 결제 여부만 보았기 때문에, `amount`가 없거나 음수인 결제 주문은 이제 세지 않습니다. 이런 데이터가 실제로 있는지는 노트와 diff만으로는 알 수 없습니다. 데이터를 직접 확인해 보는 것이 좋습니다.

질문

노트 아래에 남은 질문과 답(haiku가 실제로 쓴 글)

대화창에서 Claude에게 그냥 묻기 1.6.0 새 기능

명령을 외우지 않아도 됩니다. 대화창에 "오늘 배운 거 알려 줘", "복습할 개념 뭐야?", "const 노트 찾아 줘"처럼 물으면, Claude가 learn-notes의 학습 기록 도구를 불러 내 노트와 개념 모음을 읽고 답합니다. 화면에는

`learn-notes - notes (MCP)` 도구 줄이 보입니다.

무엇을 읽나요

이 프로젝트의 노트(찾기 · 한 기간 · 노트 하나와 그 아래 질문), 모든 프로젝트의 배운 개념, 지금 복습할 개념과 오늘 뭣, 학습 기록. 한 번에 4,000자까지 넘깁니다.

안심하고 쓰세요

읽기 전용이라 권한 창이 뜨지 않고, 기록을 바꾸지 않습니다. learn-notes 쪽 모델 호출도 없습니다. 답은 대화 중인 Claude가 하므로 평소 대화처럼 메인 세션 사용량에 들어갑니다.

노트 아래 질문(i)과 무엇이 다른가요?

노트 아래 질문은 그 노트와 바뀐 코드만 보고 저렴한 모델이 짧게 답합니다. 대화창 질문은 지금 대화 중인 Claude가 여러 노트와 기록을 모아 답합니다. /learn 도움말 맨 위에 "대화창에 그냥 물어봐도 됩니다" 줄이 보이면 이 도구가 켜진 것입니다.

6

내 코드로 푸는 복습 퀴즈

잊을 때쯤, 내가 만든 코드로 다시 떠올립니다.

상태줄에 학습 노트 · 오늘 복습 3개 · /learn 복습 이 보이면, 말 그대로 /learn 복습 을 칩니다. 패널이 퀴즈로 열리며 문제를 받습니다(한 번에 최대 3문제). 패널에서는 **q**, 퀴즈 보기(**4**)에서는 **s**로 받아도 됩니다. 복습할 때가 된 개념부터, 되도록 서로 다른 노트에서 골라 내 코드를 보여 주며 묻습니다.

문제는 세 가지 유형

문제 제목에 유형이 붙습니다. 예: 문제 1/3 · 예측 .



예측

코드를 보여 주고 출력이나 값을 맞춥니다. 값이 맞으면 설명이 없어도 맞습니다.



왜

어떤 줄을 빼거나 바꾸면 어떻게 되는지, 왜 그렇게 썼는지 묻습니다.



바꿔 보기

바라는 동작을 말하고 어느 줄을 어떻게 바꿀지 묻습니다. 동작이 같으면 다른 코드도 맞습니다.

답을 적고 Enter만 누르면 이어집니다

① 문제

커서가 내 답 칸에
있습니다

② 적기

한글로 그냥 적고 Enter

③ 채점

"채점하는 중입니다..." 뒤
판정

④ 다음

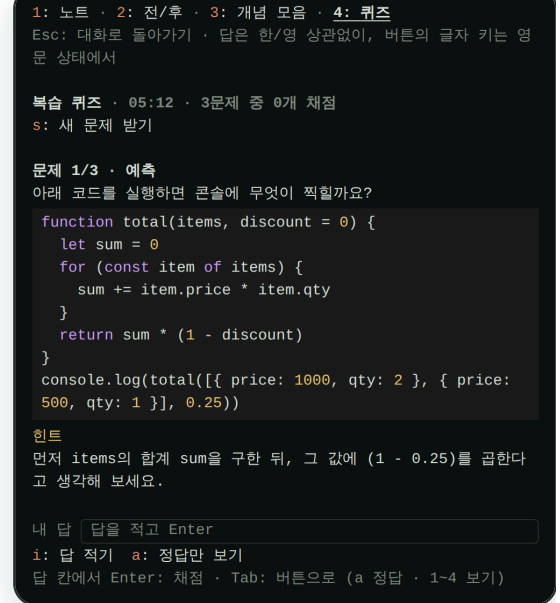
커서가 다음 문제 칸으로

⑤ 끝

마지막 뒤에는 s: 새 문제
받기

퀴즈를 푸는 동안 한/영을 바꿀 필요가 없습니다. Claude는 ✓ 맞힘 · △ 거의 맞음 · ✕ 틀림 중 하나로 채점하고, 내 답에서 맞은 점과 빠진 점을 한두 문장으로 짚어 줍니다. 표현이 달라도 뜻이 같으면 맞습니다.

키	하는 일
Enter	내 답 칸에 적은 답을 Claude에게 채점받기. 칸에서는 글자가 답으로 들어가니(한글로 적어도 됩니다), 아래 키는 Tab 으로 버튼에 간 뒤 누릅니다
h	힌트. 답을 말하지 않고 떠올릴 실마리만 보여 줍니다
a	정답만 보기. 그다음 o 맞혔어요 · x 틀렸어요로 스스로 채점
f	채점 바꾸기. Claude의 채점이 이상하면 맞힘 ↔ 틀림
n	이 개념을 배운 노트로. Claude가 틀림·거의로 채점한 문제 아래에 나옵니다. 그 노트에서 r 로 따라가 보세요
s	문제 받기 · 새 문제 받기. 풀던 문제가 남았으면 한 번 더 눌러야 받습니다



내 코드로 낸 문제와 힌트(h)

답 칸에 커서가 있을 때

글자 키는 모두 칸에 적합합니다(한글로 적어도 됩니다). **h** · **a** · **n** · **f**는 **Tab**으로 버튼에 간 뒤 누르거나 마우스로 누르세요. 칸 아래에 흐리게 "답 칸에서 Enter: 채점 · Tab: 버튼으로"가 보입니다. 노트가 패널에 없으면 **n** 대신 **/learn** 찾기 개념 이름을 안내합니다.



적은 답을 Claude가 채점하고, 맞은 점과 빠진 점을 짚었습니다. 채점이 이상하면 **f**로 바꿉니다. 복습 단계도 처음부터 그렇게 채점한 것처럼 돌아갑니다.

간격 반복 복습 잊을 때쯤 다시

개념마다 복습 단계가 있습니다. 때가 된 개념을 퀴즈로 떠올려 맞힐 때마다 다음 복습까지의 간격이 늘어납니다.

1일

0단계

3일

1단계

7일

2단계

14일

3단계

30일

4단계

60일

5단계



직접 적어 맞으면

한 단계 올라 더 나중에 나옵니다. 때가 되기 전이나 같은 날 다시 맞힌 것은 단계를 그대로 둡니다.



힌트나 정답을 보고 맞으면

결과 줄이 ☒ 맞힘 · ☐ 힌트 봄 또는 ☐ 정답 봄 이 되고, 간격은 그대로입니다. 정답을 본 뒤 ☐ 로 맞힘을 골라도 같습니다.



거의 맞으면

한 단계 내려가 조금 더 빨리 다시 나옵니다.



틀리면

처음 단계로 돌아가 다음 퀴즈 맨 앞에 나옵니다.

떠올린 것만 복습으로 칩니다

노트에서 다시 만난 개념은 다시 만난 개념 ×3처럼 보이지만, 복습 날짜를 뒤로 밀지 않습니다. 퀴즈로 떠올려야 다음 단계로 갑니다. 퀴즈를 아직 안 본 개념은 처음 만난 다음 날(두 번 이상 만났으면 3일 뒤) 복습에 올라옵니다. 간격을 늘리는 길은 **답을 직접 적어 채점받는 것**입니다.

오늘 복습은 하루 10개까지

상태줄은 오늘 뭇만 썬니다. 오늘 채점한 문제가 10개가 되면 상태줄이 내려가고, 퀴즈 끝에 "오늘 복습을 마쳤습니다 · 남은 개념은 내일 나옵니다"가 나옵니다. 전체 복습 대상 수는 개념 모음과 [/learn 기록](#) 에서 봅니다. 상태줄을 끄려면 설정의 [reviewReminder](#) 를 끕니다.

졸업과 아는 개념

마지막 단계(60일)에서 때가 되어 또 직접 맞히면 졸업합니다. 퀴즈 끝에 **졸업: 구조 분해 할당** 이 나옵니다. 이미 아는 개념은 [/learn](#) **안다** 이름 (심표로 여럿)으로 표시하고, [/learn](#) **모른다** 이름 으로 되돌립니다.

아는 개념은 오늘 복습 수 · 퀴즈 · 개념 목록에서 빠지고, 개념 모음 맨 아래 **아는 개념 12개 · ...** 한 줄로 접힙니다. 다음 노트부터는 배울 개념에 넣지 않습니다.

무엇을 배웠는지, 얼마나 꾸준했는지 눈에 보입니다.

개념 모음 패널에서 3

노트에 "배운 개념"을 프로젝트를 넘어 한 목록에 모읍니다. 맨 위 한 줄에 **배운 개념 12개 · 최근 7일 새로 3 · 연속 3일째 · 퀴즈 4/6** 이 나오고, 그 아래에 복습할 개념(틀린 것 먼저)이 나옵니다.

개념마다 만난 횟수, 최근에 만난 날, 복습 상태, 최근 설명, 그 개념이 나온 노트(누르면 그 노트로)가 있습니다. 복습 상태는 **아직 떠올려 본 적 없음** · **다음 복습 3일 뒤** · **퀴즈 틀림 중** 하나입니다.

표기만 다른 이름은 한 개념으로 셉니다. 그래도 갈라졌다면 `/learn` 합치기 `A = B` 로 합치고, 거꾸로 하면 되돌립니다.

1: 노트 · 2: 전/후 · 3: 개념 모음 · 4: 퀴즈 • q: 복습 1개
Esc: 대화로 돌아가기 · 글자 키는 영문 상태에서

배운 개념 4개 · 최근 7일 새로 4 · 연속 1일째

복습할 개념 1개 · 틀린 것 먼저, 잊을 때쯤 다시 나옵니다 · q로 복습
소수로 적는 비율 · 1일 지남

모든 개념 · 여러 번 만난 순

for...of ×2 · 아직 떠올려 본 적 없음 · 최근 오늘
배열의 값을 하나씩 꺼내 쓰는 반복문입니다. 순번(인덱스)을 따로 세지 않아도 됩니다 - `for (const order of orders) {`
노트: 10-01 06:12 cart.js 05:12 orders.js

기본값 매개변수 ×2 · 아직 떠올려 본 적 없음 · 최근 오늘
인자를 생략했을 때 쓸 값을 미리 정해 두는 문법입니다. 여기서는 0입니다 - `function countPaid(orders, minAmount = 0) {`
노트: 10-01 06:12 cart.js 05:12 orders.js

복합 대입 연산자 ×1 · 아직 떠올려 본 적 없음 · 최근 오늘
현재 값에 더한 결과를 변수에 바로 저장하는 줄임 표기입니다. `n += 1`은 `n = n + 1`과 같습니다 - `n += 1`
노트: 05:12 orders.js

소수로 적는 비율 ×1 · 아직 떠올려 본 적 없음 · 최근 10-01
10%를 0.1처럼 1보다 작은 소수로 나타내는 방식입니다 - `return sum * (1 - discount)`
노트: 10-01 06:12 cart.js

개념 모음 보기(3)

/learn 기록 한곳에 모은 학습 기록

연속 학습일(가장 길었던 날 수), 최근 7일의 노트 · 새 개념 · 다시 만난 개념 · 퀴즈, 최근 30일 정답률, 복습할 개념과 오늘 몫, 최근 7일 노트 수를 요일별로 한 줄, 배운 개념(많이 만난 순 20개), 아는 개념을 보여 줍니다. 맨 끝에는 learn-notes가 부른 모델 호출 수가 붙습니다. 모든 프로젝트를 합쳐 최근 120일을 남깁니다. 1.5까지의 `/learn stats` · `concepts` 도 같은 답을 냅니다.

/learn 보고서 1.6.0 새 기능 · 코드 없는 학습 보고서

멘토와의 1:1, PR 설명, 팀 채널에 붙일 수 있는 짧은 보고서를 만듭니다. 기본은 최근 7일이고, 이번주 · 어제 · 오늘 · 2026-10-03 도 됩니다. 보고서는 **클립보드에 복사**되고, 자동 저장(autoSave)이 켜져 있으면 일지 풀더에 **report-첫날_끝날.md** 로도 남습니다. 모델 호출은 없습니다.

학습 보고 · 2026-10-03 ~ 2026-10-09 (최근 7일)

모든 프로젝트를 합친 기록입니다. 코드 · 요청 문장 · 파일 이름은 들어 있지 않습니다.

- 학습한 날 5일 · 지금 연속 3일째
- 학습 노트 23개
- 새로 배운 개념 9개: 구조 분해 할당 · `async/await` · ...
- 다시 만난 개념 6개
- 퀴즈 24문제 중 18개 맞힘 (75%)
- 다시 볼 개념(퀴즈에서 틀림): 클로저 · `Promise.all`
- 지금 복습할 개념 4개
- 설명 수준 beginner
- 학습 노트의 모델 호출 61번 (자동 노트 40) · 입력 약 18만 · 출력 약 3만 토큰

이번 기간에 배운 것을 내 말로 한 줄

맨 끝 빈 칸에 이번 기간에 배운 것을 내 말로 한 줄 적어 보세요. 개념 이름에 업무 내용이 드러날 수 있으니 붙이기 전에 한 번 읽어 보세요. 클라우드 세션이나 휴대폰(원격)처럼 복사할 수 없는 곳에서는 보고서 전문을 대화창에 적습니다.

하루 · 한 주 정리와 그 밖의 기록

/learn 정리

오늘 노트를 모두 읽고 **한 일 · 핵심 개념 · 헛갈리기 쉬운 것 · 다음에 해 볼 것**으로 정리합니다. 헛갈리기 쉬운 것은 그 기간에 퀴즈에서 틀린 개념부터 다룹니다. **어제 · 이번주 · 최근 7일** · 날짜도 됩니다.

/learn 일지

오늘 노트의 목차를 봅니다. **어제 · 10-02** 같은 날짜도 되고, **/learn 일지 목록**은 일지가 있는 날짜를 보여 줍니다.

/learn 안키

퀴즈 문제(최근 300개)와 개념을 Anki 카드 파일로 씁니다. Anki 데스크톱에서 파일 → 가져오기로 고르면 **learn-notes** 데크에 들어갑니다.

파일은 어디에 있나요

무엇	어디에
날짜별 일지	<code>~/ .claude/learning-notes/2026-10-03_shop_xxxxx.md</code> · 노트, 바뀐 코드, 질문과 답, 정리
개념 모음 표	같은 폴더의 <code>concepts.md</code>
학습 보고서	같은 폴더의 <code>report-2026-10-03_2026-10-09.md</code>
Anki 카드	같은 폴더의 <code>learn-notes-anki.txt</code>
지난 노트 · 퀴즈 · 기록	Claude Code의 플러그인 저장소(프로젝트마다 최근 노트 30개)

명령 한눈에 보기

한글로 씁니다. 영어(quiz · recap · ask · stats · report · find · day ...)도 그대로 됩니다.

명령	하는 일
<code>/learn</code>	패널 열기. 키는 대화 입력칸에 둡니다(패널로는 ctrl+x tab · 클릭). 패널을 띄울 수 없는 화면이면 마지막 노트를 대화창에
<code>/learn 복습</code>	오늘 복습 바로 시작. 패널을 퀴즈로 열고 커서를 답 칸에 둡니다
<code>/learn 퀴즈</code>	풀던 퀴즈를 다시 보여 줍니다(모델 호출 없음). 풀던 문제가 없으면 복습할 개념으로 새 문제
<code>/learn 퀴즈 새로</code>	언제나 새 문제 받기
<code>/learn 퀴즈 2 내 답</code>	2번 문제를 Claude에게 채점받기. 번호 없이 적으면 첫 번째 안 푼 문제의 답으로 채점합니다
<code>/learn 퀴즈 힌트</code>	지금 문제 하나의 힌트 (<code>힌트 2</code> 면 2번)
<code>/learn 퀴즈 정답</code>	지금 문제 하나의 정답 (<code>정답 3</code> 이면 3번). 채점은 하지 않습니다
<code>/learn 퀴즈 맞음 1</code>	정답을 본 뒤 스스로 채점. <code>틀림 2</code> 도 됩니다. 이미 채점한 문제도 바꿉니다
<code>/learn 정리</code>	오늘 배운 것 정리 (어제 · 이번주 · 최근 7일 · 날짜)
<code>/learn 질문 ...</code>	패널에서 고른 노트(없으면 마지막 노트)에 대해 묻기
<code>/learn 기록</code>	연속 학습일 · 정답률 · 복습할 개념 · 배운 개념 · 모델 호출 수
<code>/learn 보고서</code>	코드 없는 학습 보고서를 클립보드와 파일로 (최근 7일 · 이번주 · 어제 · 오늘 · 날짜)
<code>/learn 찾기 말</code>	모든 프로젝트의 노트에서 요청 · 내용 · 파일 이름 · 개념으로 찾기
<code>/learn 일지</code>	오늘 노트 목차 (어제 · 날짜). <code>/learn 일지 목록</code> 은 일지가 있는 날짜
<code>/learn 도움말</code>	명령 목록

가끔 쓰는 것

명령	하는 일
<code>/learn 마지막</code>	마지막 노트를 대화창에
<code>/learn 합치기 A = B</code>	이름만 다른 같은 개념 합치기 (거꾸로 하면 되돌림)
<code>/learn 안다 이름</code>	아는 개념으로 표시해 복습에서 빼기 (선택표로 여럿). <code>/learn 모른다 이름</code> 으로 되돌림
<code>/learn 안키</code>	퀴즈 문제와 개념을 Anki 카드 파일로

/learn 비우기

이 프로젝트의 노트를 패널에서 비웁니다. 처음에는 몇 개를 비울지 안내만 하고, `/learn 비우기 확인` 이라야 지웁니다. 일지와 개념 모음은 그대로

패널이 안 보이는 곳에서도

클라우드 세션이나 모바일 앱처럼 패널이 안 보이는 곳에서는 명령만으로 다 할 수 있습니다. `/learn` 퀴즈로 문제를 받고, `/learn` 퀴즈 1 내 답으로 채점받고, `/learn` 질문으로 묻습니다. 대화창에 그냥 물어봐도 됩니다.

9

설정 · 비용 · 개인정보

`/config`의 `learn-notes` 항목에서 바꿉니다. 관리자가 잠근 항목은 바꿀 수 없습니다(10장).

항목	기본값	설명
<code>autoNote</code>	켜짐	끄면 바뀐 코드만 모으고, 노트는 <code>w</code> 를 눌렀을 때만 씁니다
<code>model</code>	haiku	haiku · sonnet. 노트 · 퀴즈 · 채점 · 질문에 씁니다. haiku가 가장 쌉니다
<code>level</code>	beginner	beginner(용어를 쉬운 말로) · intermediate(개념 위주) · advanced(설계 · 트레이드오프)
<code>autoOpen</code>	켜짐	첫 노트가 생기면 오른쪽에 패널을 엽니다(전체 화면일 때)
<code>reviewReminder</code>	켜짐	프롬프트 아래에 오늘 복습할 개수를 띄웁니다
<code>autoSave</code>	켜짐	날짜별 마크다운 일지에 씁니다. 끄면 파일 없이 패널에만 남습니다
<code>saveDir</code>	(비움)	비우면 <code>~/.claude/learning-notes</code>
<code>excludePaths</code>	(비움)	새 · 노트에서 뺄 파일 패턴, 쉼표로 구분 (예: <code>legacy/**, *.generated.ts</code>)
<code>dailyAutoNotes</code>	0	새 · 하루 자동 노트 최대 개수. 0이면 제한이 없습니다

비용

파일을 바꾼 때마다 노트 호출이 **한 번** 일어납니다(기본 haiku). 띄어쓰기 · 줄바꿈만 바뀐 턴(JS · TS · CSS · JSON · HTML 등)과 비밀 · 잠금 파일만 바뀐 턴은 부르지 않습니다. 퀴즈 받기 · 답 채점 · 질문 · 따라가기 · 더 쉽게 · 정리는 할 때마다 한 번씩입니다. 기록 · 보고서 · 개념 모음 · 안키 · 대화창 도구는 부르지 않습니다.

얼마나 썼는지

`/learn` 기록 맨 끝에 학습 노트의 모델 호출: 오늘 12번 (자동 노트 9) · 최근 7일 61번 · 입력 약 18만 · 출력 약 3만 토큰 이 붙습니다. 금액은 계정마다 달라 적지 않습니다. 노트 호출은 메인 세션과 같은 계정의 요청 한도를 나눠 씁니다.

무엇이 어디로 가나요

바뀐 코드, 내 요청, Claude의 마지막 설명, 노트 질문이 모델 호출에 실려 지금 쓰는 Claude Code 계정으로 나갑니다. 노트 · 일지 · 기록은 내 컴퓨터에만 저장됩니다. 설정 없이도 아래 두 가지가 기본으로 동작합니다.



내용을 모으지 않는 파일

비밀 파일: `.env` · `.env.local` (`.env.example` 등은 제외), `*.pem` · `*.key` · `*.tfvars`, `id_rsa*`, `.npmrc` · `.git-credentials`, `*credentials*.json`, `secrets.*` 등.
잠금 · 생성 파일: `package-lock.json` · `yarn.lock` · `pnpm-lock.yaml` 같은 잠금 파일, `*.min.js` · `*.map`, `node_modules/`, 맨 위의 `dist/` · `build/` · `.next/` 등.
`excludePaths` 와 팀 규칙 파일의 `## 빼기`, 조직 설정에서 읽기가 막힌 파일도 같습니다.



가리는 값 «가림»

다른 파일의 바뀐 코드, 내 요청, Claude의 설명, 노트 질문에 든 흔한 키 형식을 «가림» 으로 바꿉니다. `sk-ant-...` · `sk-...`, `AKIA...`, `ghp_...` · `github_pat_...`, `xox?-...`, `AIza...`, `sk_live_...`, `glpat-...`, `JWT`, `PRIVATE KEY` 블록, 접속 주소의 비밀번호 (`postgres://user:비번@`), `Bearer` 토큰, 따옴표로 감싼 `password` · `secret` · `token` 값, `API_KEY=...` 같은 대문자 `env` 줄의 값, `npm_...` · `hf_...` 같은 토큰, 봇 토큰 · 웹훅 주소, 설정 파일(`.properties` · `.ini` · `YAML` 등)의 `password` · `secret` · `token` · `key` 값. 변수 이름은 남깁니다.

노트 머리줄에는 흐리게 · 노트에서 뺀 파일 2개 · · 비밀값 1곳 가림 이 붙고, 전/후 보기 맨 아래에 노트에서 뺀 파일: `.env` (비밀값이 들 수 있는 파일) · `package-lock.json` (잠금·생성 파일) 이 나옵니다. 이런 파일만 바뀐 턴은 노트도 모델 호출도 없습니다.

알아 두세요

흔한 형식만 잡습니다. 사내 고유 토큰, 공백이 든 따옴표 값(`password: "correct horse"`), 코드 파일 안의 따옴표 없는 값은 놓칠 수 있습니다. 그런 파일은 `excludePaths`에 넣으세요. 메인 Claude 세션은 이미 그 값을 봤습니다. `learn-notes`가 막는 것은 **추가 모델 호출과 디스크 · 저장소에 남는 사본**입니다. 1.6.0 전에 쓴 일지 파일은 고치지 않습니다.

모으지 않는 것

Claude가 스스로 쓰는 계획·메모리 파일, 프로젝트 밖 임시 폴더의 작업 파일, `git stash` · `checkout` · `pull`처럼 디스크에 되돌리거나 가져온 내용은 노트에 담지 않습니다. 내가 배울 코드가 아니기 때문입니다.

같은 용어로 배우고, 기본값은 관리자가 정하고, 비용은 한도로 묶습니다.

팀 규칙 · 용어 파일 `.claude/learn-notes.md`

저장소 맨 위의 `.claude/learn-notes.md` 에 팀 규칙과 용어를 적어 커밋합니다. 새 명령은 없습니다. 팀원 모두의 노트가 이 파일을 따릅니다.

```
## 규칙
- API 호출은 fetcher로 감쌉니다
- 컴포넌트 파일 이름은 PascalCase로 씁니다

## 용어
- **구조 분해 할당**: 값을 꺼내 담는 문법
- **커스텀 훅**: use로 시작하는 재사용 함수

## 빼기
- legacy/**
- `docs/old drafts/` 공백이 든 경로는 백틱
```

- **규칙**: 바뀐 코드와 직접 닿을 때만 노트의 "왜 이렇게 바꿨을까" 끝에 `팀 규칙: ...` 한 줄이 붙습니다. 달라 보이면 `팀 규칙과 다를 수 있음: ...` 으로 적어 단정하지 않습니다. 노트 아래 질문(i)도 규칙을 근거로 답합니다.
- **용어**: 배울 개념을 용어집의 이름 그대로 써서, 팀원 모두의 개념 모음에 같은 이름으로 쌓입니다.
- **빼기**: `excludePaths` 에 더해집니다.

제목은 영어(`## Rules` · `## Terms` · `## Exclude`)도 됩니다. 규칙과 용어는 합쳐 3,000자까지 싣고, 파일이 바뀌면 다음 턴에 다시 읽습니다. 노트가 아직 없는 패널에 이 저장소의 팀 규칙 파일을 읽었습니다 (`규칙 2 · 용어 2 · 빼기 2`)가 흐리게 보이면 읽힌 것입니다. 규칙 판정은 코드 리뷰를 대신하지 않습니다.

저장소에서 팀원에게 자동으로 켜기

```
// 저장소의 .claude/settings.json
{
  "extraKnownMarketplaces": {
    "learn-notes": { "source": {
      "source": "github",
      "repo": "Owen-YC/learn-notes" } }
  },
  "enabledPlugins": {
    "learn-notes@learn-notes": true
  }
}
```

이 파일을 커밋하면 저장소를 연 팀원에게 `learn-notes`가 따로 설치하지 않아도 켜집니다. 팀원이 그 폴더의 **신뢰** **창을 받아들인** 뒤부터입니다.

설정값(`level` · `model` · `excludePaths` · `dailyAutoNotes`)은 저장소 설정으로 정할 수 없습니다. Claude Code는 플러그인 설정을 사용자 설정과 관리 설정에서만 읽습니다.

관리자: 조직 전체에 설치하고 기본값 잠그기

조직의 **관리 설정**(managed settings)에 위 두 키와 `pluginConfigs` 를 넣으면 모든 사람에게 설치되고, 넣은 설정값은 **잠깁니다**. 잠긴 항목은 `/config` 의 `learn-notes`에 보이지만 바꿀 수 없고, 넣지 않은 항목은 각자 고릅니다.

```
"pluginConfigs": {
  "learn-notes@learn-notes": {
    "options": {
      "model": "haiku",
      "level": "beginner",
      "dailyAutoNotes": 40,
      "excludePaths": "legacy/**"
    }
  }
}
```

나눠 주는 길

claude.ai 조직 설정 > Claude Code > Managed settings(Teams · Enterprise), MDM, 또는 `managed-settings.json`

Windows `C:\Program Files\ClaudeCode\`

macOS `/Library/Application`

Support/ClaudeCode/

Linux · WSL `/etc/claude-code/`

마켓플레이스에 `"autoUpdate": true` 를 두면 새 판이 나왔을 때 각자 업데이트하지 않아도 따라옵니다.

조직의 읽기 금지 규칙을 따릅니다

`permissions.deny` 에 `Read(./config/prod.yml)` 처럼 읽기를 막은 파일은 노트 · 일지 · 모델 호출에 내용이 실리지 않고, 노트에서 `뺨` 파일: `config/prod.yml` (조직 설정으로 읽기가 막힌 파일) 로 표시됩니다. 대화창 학습 기록 도구를 막으려면 `mcp_learn-notes_notes` 를 `deny`에 넣습니다.

하루 자동 노트 한도

`dailyAutoNotes` 를 예를 들어 40으로 두면, 그날 자동 노트가 40개에 닿은 뒤에는 턴이 끝나도 노트를 쓰지 않습니다. 노트 칸에 `오늘 자동 노트 한도(40개)에 닿았습니다 · w로 쓰면 씹니다` 가 뜨고, 알림은 하루 한 번만 뜹니다. `w · e · r · i` · 퀴즈처럼 직접 누른 학습 동작은 막지 않습니다.

보고서로 나누기

`/learn` 보고서로 코드 없는 학습 기록을 1:1 · PR · 팀 채널에 붙여 넣을 수 있습니다(7장). 본인 회고와 멘토링용으로 쓰고, 사람을 줄 세우는 데는 쓰지 않길 권합니다.

11

문제 해결

노트가 안 생기면 위에서부터 하나씩 확인해 보세요.

- ☐ **그 턴에 파일이 바뀌었나요?** 질문에 답만 한 턴은 노트가 없습니다. Claude가 파일을 만들거나 고친 턴에만 생깁니다.
- ☐ **비밀 · 잠금 · 생성 파일만 바뀌었나요?** `.env` · `package-lock.json` 이나 `excludePaths` 에 맞는 파일만 바뀐 턴은 노트를 쓰지 않습니다(9장).
- ☐ **"띄어쓰기·줄바꿈만 바뀌어 노트를 쓰지 않았습니다"가 보이나요?** 모양만 바뀐 턴은 모델을 부르지 않습니다. 그래도 노트가 필요하면 `w` 로 씹니다.
- ☐ **"오늘 자동 노트 한도에 닿았습니다"가 보이나요?** `dailyAutoNotes` 한도입니다. `w` 를 누르면 그 노트를 씹니다. 내일이면 다시 저절로 씹니다.
- ☐ **패널이 열려 있나요?** `/learn` 으로 엽니다. 대화 오른쪽에 붙으려면 `/tui fullscreen` (전체 화면)이어야 합니다. 패널을 띄울 수 없는 화면에서는 노트가 준비될 때 대화창에 흐린 줄 `학습 노트 · 한 줄 요약 · /learn으로 보기` 가 남습니다(Claude에게는 가지 않습니다).

- ☐ **단축키가 안 먹나요?** 먼저 한/영을 확인하세요. 숫자 키는 한/영 상관없이, 글자 키는 영문 상태에서 눌립니다.
`/learn` 은 패널만 열고 키는 대화에 둡니다. `ctrl` + `x` `tab` 이나 클릭으로 패널을 고르세요. 답 칸에 커서가 있으면 글자는 칸에 적히니 `Tab` 으로 버튼에 갑니다.
- ☐ **/learn을 모른다고 하나요?** 다른 폴더에서 설치했을 수 있습니다. `/plugin` 의 Installed 탭에서 learn-notes를 확인하고, 없으면 설치 두 줄을 다시 입력합니다.
- ☐ **노트 칸이 빨간 글씨인가요?** 이유가 적혀 있습니다. "서버가 붐빕니다" · "요청 한도에 걸렸습니다"는 잠시 뒤 `w` 로 다시 쓰고, "모델을 부를 수 없습니다"는 `/config` 에서 다른 모델을, "로그인이 필요합니다"는 `/login` .
- ☐ **Windows에서 파일이 엉뚱한 곳에 생기나요?** Claude Code를 작업할 폴더에서 켜세요(`cd ~\practice` 뒤 `claude`).
- ☐ **업데이트가 안 보이나요?** `/plugin` 에서 Update 뒤 `/reload-plugins` . `/learn` 도움말 에 `/learn` 보고서 가 보이면 1.6.0입니다.

1.5에서 올라온 분께

`v` 대신 숫자 `1` ~ `4` 로 보기를 고릅니다. `/learn stats · concepts`는 `/learn` 기록 하나가 되었고, `/learn save`는 안내만 합니다(파일에 쓰지 못한 노트는 다음 노트를 저장할 때 함께 저장됩니다). `/learn` 비우기는 확인을 붙여야 지웁니다. 모델 선택지에서 opus가 빠져, opus를 골랐다면 haiku로 돌아갑니다. 퀴즈를 본 적 없는 오래된 개념이 업데이트한 날 한꺼번에 복습 대상이 될 수 있지만, 상태줄은 하루 10개까지만 셉니다.

그래도 안 되면

화면을 캡처해 github.com/Owen-YC/learn-notes/issues에 남기거나 팀 채널에 공유해 주세요. Claude Code 버전(`claude --version`)과 운영체제를 함께 적어 주시면 빨리 찾을 수 있습니다.

팀에서 이렇게 써 보세요

하루 5분, 한 주 15분이면 충분합니다.



아침: 오늘 복습

상태줄에 "오늘 복습 3개"가 보이면 `/learn 복습`. 커서가 답 칸에 있으니 답을 적고 Enter만 누르면 됩니다. 직접 적어 맞혀야 간격이 늘어납니다.



작업 중: 모르면 바로 질문

낮선 개념은 `i` 로 묻고, 코드가 어떻게 도는지 궁금하면 `r`, 어려우면 `e`. 여러 노트를 묶어 묻고 싶으면 대화창에 "이번 주에 배운 것 정리해 줘"라고 물어보세요.



하루 끝: 오늘 정리

`/learn 정리` 로 오늘 배운 것을 네 칸으로 정리합니다. "핵심 개념" 한 줄을 팀 채널에 공유해 보세요.



주 1회: 회고

`/learn 정리 이번주` 로 한 주를 돌아보고, `/learn 보고서 이번주` 에 내 말로 한 줄을 채워 멘토와의 1:1이나 팀 채널에 붙입니다.

눈높이 맞추기

처음에는 설정 level을 **beginner**로 두고, 노트가 너무 쉽게 느껴지면 **intermediate**로 올리세요. 이미 아는 개념은 `/learn 안다` 이름으로 표시하면 복습과 노트에서 빠져 새 개념에 자리를 내줍니다.

빠른 참조 · 책상 옆에 두고 보세요

오늘 첫 요청부터 노트가 쌓입니다.

설치 (Claude Code 안에서)

```
/plugin marketplace add Owen-YC/learn-notes  
/plugin install learn-notes@learn-notes
```

다시 시작 → `/tui fullscreen` → 평소처럼 코딩 요청 → `/learn` 으로 패널

패널 (ctrl+x tab 또는 클릭)

- 1** **2** **3** **4** 노트 · 전/후 · 개념 · 퀴즈
- q** 오늘 복습 바로 시작
- p** **n** 이전 · 다음 노트
- w** 다시 쓰기
- t** 이 노트 퀴즈
- e** 더 쉽게
- r** 예시로 따라가기
- i** 질문하기
- Esc** 대화로 돌아가기

퀴즈 (커서가 답 칸에)

- Enter** 적은 답 채점받기
 - h** 힌트
 - a** 정답만 보기
 - o** **x** 맞혔어요 · 틀렸어요
 - f** 채점 바꾸기
 - n** 이 개념을 배운 노트
 - s** 새 문제 받기
- 숫자 키는 한/영 상관없이, 글자 키는 영문 상태에서

자주 쓰는 명령

`/learn 복습`

오늘 복습 바로 시작

`/learn 정리`

오늘 배운 것 정리

`/learn 기록`

연속 학습일 · 정답률 · 개념

`/learn 도움말`

전체 명령

`/learn 퀴즈 1 내 답`

Claude가 채점

`/learn 질문 ...`

노트에 대해 묻기

`/learn 보고서`

코드 없는 학습 보고서

"오늘 배운 거 알려 줘"

대화창에 그냥 묻기